Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_01.pdf`

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

This review tutorial covers the fundamental concepts of the module before moving into deeper sessions. By the end of this tutorial, the student will be able to:

• Situate the different themes of the module (architecture, OS, algorithms, networks, security, Python)

• Apply the basic concepts seen in the introduction

• Establish connections between the different parts of the program

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — Architecture	15 min	20 min
Exercise 2 — Processes	18 min	38 min
Exercise 3 — Algorithm	18 min	56 min
Exercise 4 — Networks	18 min	74 min
Exercise 5 — Python	16 min	90 min
Total	90 min	

Conceptual module map

```
mindmap
root((Computer Science))
Architecture
Von Neumann
CPU & ALU
Memory hierarchy
Operating Systems
Processes & threads
Memory management
File system
Algorithms
Variables & types
Conditions & loops
Complexity
Data Structures
```

```
Arrays
Stacks & queues
Linked lists
Networks
LAN / WAN
IP / DNS
Client-server
Security
CID
Passwords
GDPR
Python
Search & sort
Classes & objects
Dictionaries
```

Exercise overview

```
graph LR
subgraph "TD01 — Review"
A[Exercise 1<br/>Architecture] --> B[Exercise 2<br/>Processes]
B --> C[Exercise 3<br/>Algorithm]
C --> D[Exercise 4<br/>Networks]
D --> E[Exercise 5<br/>Python & Security]
end
style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#000
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff
```

Exercise 1 — Architecture and coding (15 min)

Problem statement: Explain the role of the ALU, the control unit, and main memory. Convert 13 to binary.

Solution:

- ALU: arithmetic and logical operations.
- CU: orchestrates the fetch-decode-execute cycle.
- Memory: stores instructions and data.
- $13_{10} = 1101_{2}$ (successive division by 2).

Exercise 2 — Processes and threads (18 min)

Problem statement: Distinguish between a process and a thread. Example: web browser.

Solution:

- Process: own memory space, PID.
- Thread: execution flow sharing the process memory.
- Browser: 1 process, UI, rendering, network, extensions threads.

Exercise 3 — Maximum algorithm (18 min)

Problem statement: Pseudocode for the maximum of an array. Complexity?

Solution:

```
max ← T[0]
for i from 1 to n-1 do
if T[i] > max then max ← T[i]
return max
```

Complexity: O(n) — one pass.

Exercise 4 — Networks (18 min)

Problem statement: Differentiate IP, mask, gateway. Role of DNS.

Solution:

• IP: host identifier.

• Mask: separates network/host.

• Gateway: exit to another network.

• DNS: name → IP address resolution.

Exercise 5 — Python and security (16 min)

Problem statement: Write a Python script that computes the average of 5 grades. List 3 security best practices.

Solution:

```
notes = [12, 14, 11, 15, 13]
print(sum(notes) / len(notes))
```

Security: MFA, updates, backups, unique passwords.

Self-assessment quiz

1. The CPU execution cycle starts with:

a) Execute b) Fetch ✓ c) Decode

2. A thread shares the memory of:

a) Another thread b) Its parent process ✓ c) All processes

3. Which sorting algorithm has $O(n^2)$ complexity?

a) Merge sort b) Bubble sort ✓ c) Binary search

4. DNS translates:

a) An IP to MAC b) A name to IP ✓ c) A URL to HTML

5. Which keyword defines a function in Python?

a) function b) def ✓ c) define

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)
Machine architecture	20 %	Cannot distinguish components	Components partially identified
Processes and OS	20 %	Cannot differentiate process/program	Concepts partially mastered
Algorithms	20 %	No correct algorithm	Partially correct algorithms
Networks	20 %	Does not understand client-server model	Network concepts partially understood
Python	20 %	No functional script	Partially functional script

Pedagogical Note ENS

Teaching transposition

This review tutorial shows the interconnection of the module's themes. In Year 10 (SNT), a similar tutorial can be offered but with simplified exercises: binary conversion on small numbers, device recognition, using a browser to illustrate client-server. The goal is to give an overview without going into technical details.

Trainer advice

Use this tutorial as a diagnostic assessment at the beginning of the module. Ask students to answer without preparation, then correct collectively to measure the initial level. The results will help adapt the progression of subsequent sessions.

Extension

Create a collective mind map (on paper or with a tool like XMind) that connects all the themes covered. Each student adds a concept or an example seen in the tutorial. This map will serve as a guiding thread throughout the module.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_02.pdf`

Computer Science Tutorial — ENS Year 1 — Session 02 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student will be able to:

• Convert a decimal number to binary and vice versa

• Convert between binary, decimal, and hexadecimal bases

• Decode ASCII characters from a binary sequence

• Estimate the size of different file types (text, image, audio)

• Use unit prefixes (KB, MB, GB) and calculate compression ratios

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — Binary conversions	15 min	20 min
Exercise 2 — Decimal conversions	15 min	35 min
Exercise 3 — Hexadecimal	15 min	50 min
Exercise 4 — Character encoding	15 min	65 min
Exercise 5 — File sizes	20 min	85 min
Correction	5 min	90 min
Total	90 min	

Decimal → Binary conversion

```
flowchart TD
A[Decimal number: 42] --> B{Divide by 2}
B --> C[Remainder = 0]
B --> D[Remainder = 1]
C --> E[Write down remainder]
D --> E
E --> F{Quotient = 0 ?}
F -->|No| B
F -->|Yes| G[Read remainders<br/>backwards]
G --> H[Binary result: 101010]

style A fill:#4a90d9,color:#fff
style H fill:#2ecc71,color:#fff
```

```
style B fill:#e67e22,color:#000
style G fill:#9b59b6,color:#fff
```

Storage unit hierarchy

```
graph LR
subgraph "Storage units"
bit --> Byte[1 byte = 8 bits]
Byte --> KB[1 KB = 1024 bytes]
KB --> MB[1 MB = 1024 KB]
MB --> GB[1 GB = 1024 MB]
GB --> TB[1 TB = 1024 GB]
end

subgraph "Examples"
Text["Text 5000 char.<br/>~ 5 KB"]
ImageBW["BW Image 200×150<br/>~ 3.7 KB"]
ImageC["24-bit Image 200×150<br/>~ 88 KB"]
Photo["Photo 4000×3000<br/>~ 34.3 MB"]
end

KB --> Text
KB --> ImageBW
MB --> ImageC
MB --> Photo

style bit fill:#4a90d9,color:#fff
style TB fill:#e74c3c,color:#fff
style Text fill:#2ecc71,color:#fff
style ImageBW fill:#2ecc71,color:#fff
style ImageC fill:#2ecc71,color:#fff
style Photo fill:#2ecc71,color:#fff
```

Exercise 1 — Binary conversions (15 min)

Problem statement: Convert the following numbers to binary.

1. 13 (decimal) → binary
2. 25 (decimal) → binary
3. 42 (decimal) → binary
4. 100 (decimal) → binary

Answer:

1. $13 = 8+4+1 = 1101_2$
2. $25 = 16+8+1 = 11001_2$
3. $42 = 32+8+2 = 101010_2$
4. $100 = 64+32+4 = 1100100_2$

Method: successive divisions by 2, read remainders backwards.

Exercise 2 — Decimal conversions (15 min)

Problem statement: Convert the following binary numbers to decimal.

1. 1010_2
2. 1111_2
3. 100000_2
4. 110011_2

Answer:

1. $1\times8 + 0\times4 + 1\times2 + 0\times1 = 10$
2. $1\times8 + 1\times4 + 1\times2 + 1\times1 = 15$
3. $1\times32 + 0\times16 + 0\times8 + 0\times4 + 0\times2 + 0\times1 = 32$
4. $1\times32 + 1\times16 + 0\times8 + 0\times4 + 1\times2 + 1\times1 = 51$

Exercise 3 — Hexadecimal (15 min)

Problem statement: Convert to hexadecimal.

1. 255 (decimal) → hex
2. 16 (decimal) → hex
3. $1101\ \ 1010_2$ → hex
4. $A3_{16}$ → decimal

Answer:

1. $255 = 15\times16 + 15 = FF_{16}$
2. $16 = 1\times16 + 0 = 10_{16}$
3. $1101\ \ 1010_2 = DA_{16}$
4. $A3_{16} = 10\times16 + 3 = 163$

Exercise 4 — Character encoding (15 min)

Problem statement: In ASCII (7 bits), A = 65, a = 97, space = 32. Decode the following binary sequence:

`01000001 00100000 01000010 01001111 01001110`

Hint: Each group of 8 bits = 1 character.

Answer:

- $01000001_2 = 65 = A$
- $00100000_2 = 32 =$ space
- $01000010_2 = 66 = B$
- $01001111_2 = 79 = O$
- $01001110_2 = 78 = N$

Result: `A BON`

Exercise 5 — File sizes (20 min)

Problem statement: Calculate the estimated size of the following files:

1. A text of 5000 ASCII characters (no formatting).

2. A 200×150 pixel black and white image (1 bit/pixel).

3. A 200×150 pixel 24-bit image (TrueColor).

4. A 4000×3000 pixel 24-bit photo compressed JPEG (10:1 ratio).

Answer:

1. 5000 × 1 byte = 5000 bytes ≈ 4.9 KB

2. 200 × 150 × 1 bit = 30,000 bits = 3750 bytes ≈ 3.7 KB

3. 200 × 150 × 24 bits = 720,000 bits = 90,000 bytes ≈ 87.9 KB

4. 4000 × 3000 × 3 bytes = 36,000,000 bytes ≈ 34.3 MB. JPEG compression (10:1) → ~3.4 MB

Self-assessment quiz

1. What is the decimal value of 1010_2?

a) 8 b) 10 ✓ c) 12

2. How many bits are needed to encode the number 256 in binary?

a) 8 b) 9 ✓ c) 10

3. The hexadecimal number FF_{16} equals:

a) 240 b) 255 ✓ c) 256

4. In ASCII, the character A is encoded as:

a) 65 ✓ b) 97 c) 32

5. One GB (gigabyte) equals:

a) 1000 MB b) 1024 MB ✓ c) 1024 KB

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acqui
Binary ↔ decimal conversion	20 %	Incorrect conversions	Partially correct conversions	Correc
Hexadecimal conversion	20 %	Incorrect conversions	Partially correct conversions	Correc
ASCII/UTF-8 encoding	20 %	Does not understand the principle	Partial understanding	Compl
File size calculation	20 %	Incorrect calculations	Partially correct calculations	Correc
Bitmap image	20 %	Does not understand pixel encoding	Partial understanding	Compl

Pedagogical Note ENS

Teaching transposition

Binary and hexadecimal conversions are approachable from Year 10 (SNT) using successive division or powers of 2. Binary can be introduced with card games (cards worth 1, 2, 4, 8,

16...) and having students guess a number chosen by a classmate. For file sizes, use concrete examples (phone photos, Word documents) that students handle daily.

Trainer advice

Plan progressive exercises: start with small numbers (0-15) that are easy to verify mentally, then gradually increase. For hexadecimal, emphasize going through binary (grouping by 4 bits). Exercise 5 (sizes) is very practical: ask students to open the properties of their own files to verify calculations.

Extension

Propose a mini-project: create a small Python script that automatically converts a decimal number to bases 2, 10, and 16, and displays the calculation steps. This reinvests the notions of loops and conditions seen in algorithms while consolidating base conversions.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_03.pdf`

Computer Science Tutorial — ENS Year 1 — Session 03 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student will be able to:

• Identify and describe the 5 units of the von Neumann model

• Order the steps of the instruction execution cycle (fetch-decode-execute-writeback)

• Classify different types of memory by speed and size

• Execute basic command-line commands (PowerShell)

• Explain the role of each component in the overall operation of a computer

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — von Neumann	15 min	20 min
Exercise 2 — Execution cycle	15 min	35 min
Exercise 3 — Memory hierarchy	20 min	55 min
Exercise 4 — Command line	15 min	70 min
Exercise 5 — Architecture QCM	15 min	85 min
Correction	5 min	90 min
Total	90 min	

Von Neumann model

```
block-beta
columns 3
UC["Control Unit<br/>(Control)"] space ALU["ALU<br/>(Processing)"]
block:Memory["Main memory<br/>(RAM)"]
columns 1
Instructions["Instructions"]
Data["Data"]
end
Input["Input<br/>(Keyboard)"] space Output["Output<br/>(Screen)"]

Bus["System bus"] across

UC --> Bus
```

```
ALU --> Bus
Memory --> Bus
Input --> Bus
Output --> Bus

style UC fill:#4a90d9,color:#fff
style ALU fill:#e67e22,color:#000
style Memory fill:#2ecc71,color:#fff
style Input fill:#9b59b6,color:#fff
style Output fill:#9b59b6,color:#fff
style Bus fill:#e74c3c,color:#fff
```

Execution cycle and memory hierarchy

```
flowchart LR
subgraph "Execution cycle"
direction LR
A[Fetch<br/>Loading] --> B[Decode<br/>Decoding]
B --> C[Execute<br/>Execution]
C --> D[Write-back<br/>Writing]
end

subgraph "Memory hierarchy"
direction TB
R[Registers<br/>512 B • < 1 ns] --> L1[L1 Cache<br/>64 KB • ~1 ns]
L1 --> L2[L2 Cache<br/>512 KB • ~4 ns]
L2 --> RAM[RAM<br/>16 GB • ~50 ns]
RAM --> SSD[SSD<br/>1 TB • ~100 µs]
SSD --> HDD[HDD<br/>2 TB • ~10 ms]
end

D --> R

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#000
style D fill:#9b59b6,color:#fff
style R fill:#e74c3c,color:#fff
style HDD fill:#e74c3c,color:#fff
```

Exercise 1 — Von Neumann model (15 min)

Problem statement: Name the 5 units of the von Neumann model and match each component to its role.

Components: ALU, Control unit, RAM, Hard drive, Keyboard, Screen, L1 Cache, System bus.

Unit	Associated component(s)
Processing	
Control	
Memory	
Input	
Output	

Answer:

Unit	Component(s)
Processing	ALU
Control	Control unit
Memory	RAM, L1 Cache, Hard drive
Input	Keyboard
Output	Screen

The system bus connects all units together.

Exercise 2 — Execution cycle (15 min)

Problem statement: Put the steps of the instruction execution cycle in order.

1. The control unit decodes the instruction (operation + operands)

2. The ALU executes the operation

3. The result is written to a register or to memory

4. The instruction is loaded from RAM to the instruction register

Correct order: 4 → 1 → 2 → 3 (Fetch → Decode → Execute → Write-back)

Application: Describe what happens when you press A on the keyboard (from key press to screen display). (Answer: keyboard → bus → CPU → OS → screen)

Exercise 3 — Memory hierarchy (20 min)

Problem statement: Classify the following memory types from fastest to slowest and from smallest to largest:

• SSD 1 TB

• RAM 16 GB

• L1 Cache 64 KB

• Registers 512 bytes

• HDD 2 TB

• L2 Cache 512 KB

Answer:

Level	Type	Size	Speed
1 (fast)	Registers	512 B	< 1 ns
2	L1 Cache	64 KB	~1 ns
3	L2 Cache	512 KB	~4 ns
4	RAM	16 GB	~50 ns
5	SSD	1 TB	~100 µs
6 (slow)	HDD	2 TB	~10 ms

Question: Why can a computer with 8 GB of RAM be faster than another with 16 GB?

Answer: it also depends on the CPU (frequency, cache), the SSD (vs HDD), and the OS.

Exercise 4 — Command line (15 min)

Problem statement: Write the Windows PowerShell command to:

1. List the files in the current folder
2. Create a folder `TD_Architecture`
3. Navigate into this folder
4. Create an empty file `notes.txt`
5. Display the machine's IP address

Answer:

1. `Get-ChildItem` (or `dir`)
2. `New-Item -ItemType Directory -Name "TD_Architecture"`
3. `Set-Location "TD_Architecture"` (or `cd TD_Architecture`)
4. `New-Item -ItemType File -Name "notes.txt"`
5. `Get-NetIPAddress` (or `ipconfig`)

Exercise 5 — Architecture QCM (15 min)

Problem statement: Choose the correct answer.

1. What does CPU stand for?

a) Central Processing Unit

b) Computer Personal Unit

c) Central Program Utility

2. RAM memory is:

a) Non-volatile

b) Volatile

c) As fast as registers

3. The von Neumann model stores:

a) Only data

b) Only programs

c) Data AND programs

4. The control unit is part of:

a) The hard drive

b) The processor

c) RAM

5. CPU frequency is measured in:

a) MHz or GHz

b) MB/s

c) Watts

Answer: 1a, 2b, 3c, 4b, 5a

Self-assessment quiz

1. Which unit of the von Neumann model performs calculations?

a) Control unit b) ALU ✓ c) Memory

2. The role of the cache is to:

a) Store files b) Speed up access to frequently used data ✓ c) Cool the CPU

3. The execution cycle starts with the step:

a) Execute b) Decode c) Fetch ✓

4. Among these memories, which is the fastest?

a) RAM b) Registers ✓ c) SSD

5. The command `Get-ChildItem` in PowerShell allows you to:

a) List files ✓ b) Create a folder c) Delete a file

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Von Neumann diagram	20 %	Diagram missing or wrong	Partial diagram	Complete and correct diagra
Fetch-Decode-Execute cycle	20 %	Steps in wrong order	3 steps out of 4	4 correct steps in order
Memory hierarchy	20 %	Incorrect order	3 levels out of 5	5 correct levels
CPU registers	20 %	No register identified	2-3 registers	4+ correct registers
Teaching analogies	20 %	No analogy proposed	Partial analogy	Relevant and justified analog

Pedagogical Note ENS

Teaching transposition

The von Neumann model can be introduced in middle/high school with a simple analogy: the brain (CPU) reading a recipe (program in memory), executing instructions (ALU), and using the countertop (RAM) to store temporary ingredients. Each student can mime the role of a component (memory, ALU, controller) in a theatrical activity.

Trainer advice

Exercise 2 (execution cycle) is fundamental: make sure each student can recite the 4 steps in order. Use a card game or flashcards to memorize the cycle. For memory hierarchy, emphasize the orders of magnitude (ns vs µs vs ms) which are abstract for beginners.

Extension

Have students disassemble (virtually through diagrams or actually with an old computer) a PC's components and identify them. Create a class poster representing the von Neumann model with the actual components of a lab computer.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_04.pdf`

Computer Science Tutorial — ENS Year 1 — Session 04 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student will be able to:

• Explain the role of the operating system and its components (kernel, drivers)

• Describe the states of a process and their life cycle

• Navigate a file tree and construct absolute paths

• Execute PowerShell commands to manipulate files and folders

• Distinguish kernel/user mode privileges via system calls

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — Role of the OS	15 min	20 min
Exercise 2 — Processes	15 min	35 min
Exercise 3 — File tree	20 min	55 min
Exercise 4 — PowerShell	15 min	70 min
Exercise 5 — Syscalls	15 min	85 min
Correction	5 min	90 min
Total	90 min	

Operating system architecture

```
graph TB
subgraph "User"
APP1[Application 1]
APP2[Application 2]
APP3[Application 3]
end

subgraph "User Mode"
APP1 --> API[API / System calls]
APP2 --> API
APP3 --> API
end
```

```
subgraph "Kernel Mode"
API --> Kernel[Kernel]
Kernel --> ProcM[Process management]
Kernel --> MemM[Memory management]
Kernel --> FileM[File management]
Kernel --> Drivers[Drivers]
end

subgraph "Hardware"
Drivers --> CPU
Drivers --> RAM
Drivers --> HD[Hard drive]
Drivers --> Net[Network]
end

style APP1 fill:#4a90d9,color:#fff
style APP2 fill:#4a90d9,color:#fff
style APP3 fill:#4a90d9,color:#fff
style Kernel fill:#e74c3c,color:#fff
style ProcM fill:#2ecc71,color:#fff
style MemM fill:#2ecc71,color:#fff
style FileM fill:#2ecc71,color:#fff
style Drivers fill:#2ecc71,color:#fff
style CPU fill:#e67e22,color:#000
style RAM fill:#e67e22,color:#000
style HD fill:#e67e22,color:#000
style Net fill:#e67e22,color:#000
```

Process life cycle

```
stateDiagram-v2
[*] --> Ready : Creation / Loading
Ready --> Running : Scheduling
Running --> Ready : Interrupt / Quantum end
Running --> Blocked : Blocking system call (I/O)
Blocked --> Ready : Wait finished (I/O done)
Running --> [*] : Termination

note right of Ready
Queue of
ready processes
end note

note right of Running
The CPU executes
the instructions
end note

note right of Blocked
Waiting for a resource
(disk, network, keyboard...)
end note
```

Exercise 1 — Role of the OS (15 min)

Problem statement: Match each function to its description.

Function	Description
Process management	A. Organize files in a tree structure

Memory management	B. Translate system calls to hardware
File management	C. Allocate RAM to applications
Kernel	D. Schedule program execution
Driver	E. Interface between the OS and a device

Answer: Process management → D, Memory management → C, File management → A, Kernel → B (partially), Driver → E. The kernel manages everything; drivers are part of the kernel.

Exercise 2 — Processes and states (15 min)

Problem statement: A process goes through 3 states: Ready, Running, Blocked. Match each situation to the corresponding state.

1. A process waits for the user to click a button.
2. The CPU executes the process's instructions.
3. The process is in the queue, ready to be executed.
4. A process reads a file from the hard drive.
5. The operating system has just loaded the process into memory.

Answer: 1→Blocked, 2→Running, 3→Ready, 4→Blocked (I/O wait), 5→Ready.

Exercise 3 — File tree (20 min)

Problem statement: Draw the tree structure corresponding to the following path:

```
C:\
├── Users\
│   └── Madani\
│       ├── Documents\
│       │   ├── ENS_Year1_Course\
│       │   └── TD_Info.txt
│       ├── Pictures\
│       └── Desktop\
│       └── Python_Project\
├── Program Files\
│   └── Python3\
└── Windows\
    └── System32\
```

Questions:

1. What is the absolute path to `TD_Info.txt`?
2. What is the parent folder of `Python_Project`?
3. How many folders does `C:\` contain?

Answer:

1. `C:\Users\Madani\Documents\TD_Info.txt`
2. `Desktop`
3. 3 folders (Users, Program Files, Windows)

Exercise 4 — PowerShell commands (15 min)

Problem statement: Mentally execute the following command sequence and describe the result.

```
New-Item -ItemType Directory -Name "Internship"
Set-Location "Internship"
New-Item -ItemType File -Name "Report.txt"
Set-Content -Path "Report.txt" -Value "My internship report"
Get-ChildItem
Remove-Item -Path "Report.txt"
Get-Location
```

Answer:

1. Creates the `Internship` folder
2. Goes into `Internship`
3. Creates the file `Report.txt`
4. Writes "My internship report" into the file
5. Lists the folder contents (shows `Report.txt`)
6. Deletes `Report.txt`
7. Displays the current path (`...\Internship`)

Exercise 5 — Syscalls and permissions (15 min)

Problem statement: True or False? Justify.

1. An application can directly access the hard drive without going through the OS.
2. The Linux kernel and the Windows kernel are identical.
3. A system call (syscall) allows an application to request a service from the kernel.
4. Virtual memory allows a process to use more memory than the available RAM.
5. A blocked process consumes CPU time.

Answer:

1. False — any hardware operation goes through the OS (security, abstraction).
2. False — different architectures (monolithic vs hybrid, different API calls).
3. True — this is the standard application/kernel communication mechanism.
4. True — via swap and paging, some data can be on disk.
5. False — a blocked process is not eligible for the CPU (waiting for a resource).

Self-assessment quiz

1. The kernel of an operating system is:

a) An application b) The core of the system that manages resources ✓ c) A device driver

2. A process in the "Blocked" state is waiting for:

a) The CPU to be free b) A resource (I/O, keyboard...) ✓ c) The end of the time quantum

3. In a path `C:\Users\Name\Doc\file.txt`, the parent folder of `Doc` is:

a) `C:\` b) `Name` ✓ c) `Users`

4. A system call (syscall) allows:

a) Restarting the computer b) Switching from user mode to kernel mode ✓ c) Installing software

5. The PowerShell command `Remove-Item` allows you to:

a) Create a file b) Delete a file ✓ c) Move a file

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1
Role of the kernel	20 %	Does not know the role	Role partially understood	Role well un
Process states	20 %	States in wrong order	2-3 correct states	5 correct sta
Round Robin scheduling	20 %	Does not understand the principle	Principle partially understood	Principle we
File tree	20 %	Incorrect structure	Partially correct structure	Correct stru
Command line	20 %	No correct command	2-3 correct commands	5+ correct c

Pedagogical Note ENS

Teaching transposition

The notion of an operating system can be introduced in SNT Year 10 with the analogy of an orchestra conductor (OS) directing musicians (applications) and distributing scores (resources). The file tree is a very concrete concept: students can explore Windows Explorer to find the folders seen in the tutorial. For PowerShell commands, organize a "command race" challenge where each group must complete a series of tasks as quickly as possible.

Trainer advice

Exercise 3 (file tree) is a good way to check that students understand the difference between absolute and relative paths. Offer additional exercises where students must write the relative path between two points in the tree. For process states, have students reproduce the state machine diagram from memory.

Extension

Ask students to install a Linux virtual machine (Ubuntu) under VirtualBox and reproduce the same commands in a bash terminal: `ls`, `mkdir`, `cd`, `touch`, `cat`, `rm`, `pwd`. Compare PowerShell/Linux syntaxes and discuss the philosophical differences between the two OSes.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_05.pdf`

Computer Science Tutorial — ENS Year 1 — Session 05 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student will be able to:

• Distinguish data types (integer, real, boolean, string) and use them correctly

• Write algorithms using conditional structures (if/else) and loops (for, while)

• Design a complete algorithm for calculating an average with min/max search

• Estimate and compare the algorithmic complexity of different algorithms (O(1), O(n), $O(n^2)$)

• Analyze a problem and break it down into algorithmic steps

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — Variables and types	15 min	20 min
Exercise 2 — Conditions	20 min	40 min
Exercise 3 — Loops	20 min	60 min
Exercise 4 — Complete algorithm	20 min	80 min
Exercise 5 — Complexity	10 min	90 min
Total	90 min	

Fundamental algorithmic structures

```
flowchart TD
subgraph "Data types"
T1["int: 42, -5"]
T2["float: 3.14, 0.001"]
T3["bool: true, false"]
T4["string: 'Hello'"]
end

subgraph "Control structures"
C1[Condition<br/>IF ... THEN ... ELSE]
C2[FOR loop<br/>Fixed number of iterations]
C3[WHILE loop<br/>Exit condition]
end
```

```
subgraph "Algorithms"
A1[Sum/average calculation]
A2[Min/max search]
A3[Multiplication table]
end

T1 --> C1
T2 --> C1
T3 --> C1
T4 --> C1
C1 --> A1
C1 --> A2
C2 --> A1
C2 --> A2
C2 --> A3
C3 --> A1

style T1 fill:#4a90d9,color:#fff
style T2 fill:#4a90d9,color:#fff
style T3 fill:#4a90d9,color:#fff
style T4 fill:#4a90d9,color:#fff
style C1 fill:#e67e22,color:#000
style C2 fill:#e67e22,color:#000
style C3 fill:#e67e22,color:#000
style A1 fill:#2ecc71,color:#fff
style A2 fill:#2ecc71,color:#fff
style A3 fill:#2ecc71,color:#fff
```

Complexity comparison

```
graph LR
subgraph "Complexity"
O1["O(1) — Constant<br/>Direct index access"]
On["O(n) — Linear<br/>Array traversal"]
On2["O(n²) — Quadratic<br/>Nested double loop"]
end

O1 --> Ex1["Ex: Read tab[3]"]
On --> Ex2["Ex: Search for a name"]
On2 --> Ex3["Ex: Compare each element<br/>with all the others"]

style O1 fill:#2ecc71,color:#fff
style On fill:#e67e22,color:#000
style On2 fill:#e74c3c,color:#fff
style Ex1 fill:#4a90d9,color:#fff
style Ex2 fill:#4a90d9,color:#fff
style Ex3 fill:#4a90d9,color:#fff
```

Exercise 1 — Variables and types (15 min)

Problem statement: Identify the type (integer, real, boolean, string) of each value:

1. `42`
2. `"Hello"`
3. `3.14`
4. `true`
5. `'A'`

6. `-5`
7. `0.001`
8. `"123"`

Answer:

1. Integer (int)
2. String (string)
3. Real (float)
4. Boolean (bool)
5. String (string) — a single character remains a string
6. Integer (int)
7. Real (float)
8. String (string) — quotes indicate a string even if the content is numeric

Exercise 2 — Conditions (20 min)

Problem statement: Write in pseudocode an algorithm that:

1. Asks the user for their age
2. Displays "Adult" if age ≥ 18, "Minor" otherwise
3. If age ≥ 65, also display "Senior"

Answer:

```
DISPLAY "What is your age?"
READ age
IF age >= 18 THEN
DISPLAY "Adult"
IF age >= 65 THEN
DISPLAY "Senior"
END IF
ELSE
DISPLAY "Minor"
END IF
```

Exercise 3 — Loops (20 min)

Problem statement: Write in pseudocode:

1. Display even numbers from 2 to 20 (inclusive)
2. Calculate the sum of numbers from 1 to n (n entered by the user)
3. Display the multiplication table of a number entered (from 1 to 10)

Answer:

```
// 1. Even numbers
FOR i FROM 2 TO 20 STEP 2 DO
DISPLAY i
END FOR

// 2. Sum 1 to n
```

```
DISPLAY "Enter n"
READ n
sum ← 0
FOR i FROM 1 TO n DO
sum ← sum + i
END FOR
DISPLAY "Sum: ", sum

// 3. Multiplication table
DISPLAY "Enter a number"
READ nb
FOR i FROM 1 TO 10 DO
DISPLAY nb, " × ", i, " = ", nb * i
END FOR
```

Exercise 4 — Complete algorithm (20 min)

Problem statement: Write an algorithm that:

1. Asks the user for 5 grades
2. Calculates the average
3. Displays "Passed" if average ≥ 10, "Failed" otherwise
4. Displays the highest and lowest grade

Answer:

```
max ← 0
min ← 20
sum ← 0
FOR i FROM 1 TO 5 DO
DISPLAY "Grade ", i, ": "
READ grade
sum ← sum + grade
IF grade > max THEN max ← grade END IF
IF grade < min THEN min ← grade END IF
END FOR
average ← sum / 5
DISPLAY "Average: ", average
IF average >= 10 THEN
DISPLAY "Passed"
ELSE
DISPLAY "Failed"
END IF
DISPLAY "Best grade: ", max
DISPLAY "Worst grade: ", min
```

Exercise 5 — Complexity (10 min)

Problem statement: For each algorithm, estimate the complexity (O(1), O(n), O(n²)):

1. Access the 3rd element of an array
2. Traverse an array of 100 elements
3. Compare each element of an array with all the others
4. Read the first character of a string
5. Search for a name in an unsorted list of n names

Answer:

1. O(1) — direct index access
2. O(n) — linear traversal
3. O(n²) — nested double loop
4. O(1) — direct access
5. O(n) — in the worst case, we traverse the entire list

Self-assessment quiz

1. What is the type of "42" in pseudocode?

a) Integer b) String ✓ c) Real

2. A conditional structure uses the keyword:

a) FOR b) IF ✓ c) WHILE

3. Which loop is best suited for traversing an array of known size?

a) FOR ✓ b) WHILE c) REPEAT

4. The complexity of a nested double loop (for i, for j) is:

a) O(n) b) O(n²) ✓ c) O(1)

5. In the average algorithm, finding the maximum grade requires:

a) One traversal of the array ✓ b) A complete sort c) A binary search

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Variables and types	20 %	Incorrect types	Partially correct types	Correct types
Conditional structures	20 %	Incorrect conditions	Partially correct syntax	Correct syntax
Loops	20 %	Incorrect loops	Partially correct loops	Correct loops
Complete algorithm	20 %	Algorithm missing	Partial algorithm	Complete algorithm
Algorithmic complexity	20 %	Cannot calculate	Partial calculation	Correct and justified calculation

Pedagogical Note ENS

Teaching transposition

Algorithms are at the heart of the SNT (Year 10) and NSI (Year 11/12) curricula. In Year 10, students just read and modify simple algorithms. In Year 11 NSI, they write their first algorithms in Python. The cooking recipe analogy works well: ingredients are variables, steps are instructions, and choices (if the sauce is too thin, add flour) are conditions.

Trainer advice

Exercise 4 (complete algorithm) is the most important exercise: it uses all the concepts from the tutorial. Ask students to first write the algorithm on paper, then mentally test it

with example grades (e.g., 8, 12, 15, 6, 19). Verify that each student follows the execution step by step (manual tracing). Complexity (exercise 5) is a difficult concept: explain it visually with graphs.

Extension

Ask students to translate the algorithm from exercise 4 into Python in a file `average.py`. Add a function that determines whether the average is "Excellent" (≥ 16), "Good" (≥ 14), "Fairly good" (≥ 12), "Passable" (≥ 10) or "Insufficient" (< 10). Test with different sets of grades.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_06.pdf`

Computer Science Tutorial — ENS Year 1 — Session 06 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student is able to:

• Manipulate arrays (traversal, sum, extremum search)

• Simulate the operation of a stack (LIFO) and a queue (FIFO)

• Choose the data structure adapted to a concrete problem

• Implement a parentheses checking algorithm using a stack

• Distinguish the properties and uses of each data structure

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — Arrays	15 min	20 min
Exercise 2 — Stack	20 min	40 min
Exercise 3 — Queue	20 min	60 min
Exercise 4 — Choosing a structure	15 min	75 min
Exercise 5 — Stack parentheses	15 min	90 min
Total	90 min	

Data Structures: Stack and Queue

```
graph LR
subgraph "Stack (LIFO) — Last In, First Out"
direction TB
P1["□ 30 ← pop()"] --> P2["□ 20"]
P2 --> P3["□ 10"]
P4["push(40) → □ 40"] --> P1
end

subgraph "Queue (FIFO) — First In, First Out"
direction LR
F1["□ A"] --> F2["□ B"]
F2 --> F3["□ C"]
F4["□ D"] --> F3
end
```

```
P4 -->|"push"| P1
P1 -->|"pop"| X["x ← 30"]
F1 -->|"dequeue"| Y["y ← A"]
F4 -->|"enqueue"| F3

style P1 fill:#e74c3c,color:#fff
style P2 fill:#e67e22,color:#000
style P3 fill:#4a90d9,color:#fff
style P4 fill:#2ecc71,color:#fff
style F1 fill:#2ecc71,color:#fff
style F2 fill:#4a90d9,color:#fff
style F3 fill:#9b59b6,color:#fff
style F4 fill:#e67e22,color:#000
```

Parentheses checking algorithm

```
flowchart TD
A[Start] --> B[Read expression]
B --> C[Initialize empty stack]
C --> D[For each character c]
D --> E{c = '(' ?}
E -->|Yes| F[Push '(']
F --> G[Next character]
E -->|No| H{c = ')' ?}
H -->|No| G
H -->|Yes| I{Empty stack ?}
I -->|Yes| J[Error: ) without (]
I -->|No| K[Pop]
K --> G
G --> L[No more characters ?]
L -->|No| D
L -->|Yes| M{Stack empty ?}
M -->|Yes| N[Expression correct ✓]
M -->|No| O[Error: missing )]

style A fill:#4a90d9,color:#fff
style N fill:#2ecc71,color:#fff
style J fill:#e74c3c,color:#fff
style O fill:#e74c3c,color:#fff
style F fill:#e67e22,color:#000
style K fill:#e67e22,color:#000
```

Exercise 1 — Arrays (15 min)

Problem statement: We have the following array: `notes ← [12, 15, 8, 19, 10]`

1. What is the value of the element at index 3? (Indices starting at 1)

2. Write an algorithm that computes the sum of the array elements.

3. Write an algorithm that finds the maximum value.

Answer:

1. `notes[3]` = 8 (indices start at 1 in conventional pseudo-code)

```
// 2. Sum
sum ← 0
FOR i FROM 1 TO 5 DO
sum ← sum + notes[i]
END FOR
DISPLAY "Sum: ", sum
```

```
// 3. Maximum
max ← notes[1]
FOR i FROM 2 TO 5 DO
IF notes[i] > max THEN
max ← notes[i]
END IF
END FOR
DISPLAY "Max: ", max
```

Exercise 2 — Stack (LIFO) (20 min)

Problem statement: Simulate the execution of the following operations on an initially empty stack. Indicate the state of the stack after each operation.

```
push(10)
push(20)
push(30)
x ← pop()
push(40)
y ← pop()
z ← pop()
```

Answer:

Operation	Stack after operation	Returned value
Initial	[]	—
push(10)	[10]	—
push(20)	[10, 20]	—
push(30)	[10, 20, 30]	—
pop()	[10, 20]	30
push(40)	[10, 20, 40]	—
pop()	[10, 20]	40
pop()	[10]	20

Question: What does this structure correspond to in a word processing software? (Answer: the Undo function / Ctrl+Z)

Exercise 3 — Queue (FIFO) (20 min)

Problem statement: Simulate the execution of the following operations on an initially empty queue.

```
enqueue("A")
enqueue("B")
x ← dequeue()
enqueue("C")
y ← dequeue()
enqueue("D")
z ← dequeue()
```

Answer:

Operation	Queue after operation	Returned value
Initial	[]	—
enqueue("A")	["A"]	—
enqueue("B")	["A", "B"]	—
dequeue()	["B"]	"A"
enqueue("C")	["B", "C"]	—
dequeue()	["C"]	"B"
enqueue("D")	["C", "D"]	—
dequeue()	["D"]	"C"

Question: In what real-world context is a queue used in computing? (Answer: print queue, server requests, video buffer)

Exercise 4 — Choosing a structure (15 min)

Problem statement: Which data structure (array, stack, queue, linked list) is most suitable for:

1. Browsing history (previous page)
2. A waiting line at a counter
3. Student grades in a class (fast access by student number)
4. A shopping list where items are added/removed in the middle frequently
5. Recursive function calls

Answer:

1. Stack (LIFO — going back in history)
2. Queue (FIFO — first come, first served)
3. Array (direct access by index = student number)
4. Linked list (insertion/deletion in the middle without shifting elements)
5. Stack (call stack — each call adds a stack frame, returning pops)

Exercise 5 — Algorithm with a stack (15 min)

Problem statement: Write an algorithm that checks whether a parenthesized expression is correct, for example `(3 + 2) × (5 - 1)` is correct but `(3 + 2 × (5 - 1)` is not.

Principle: use a stack. When we read `(` we push, when we read `)` we pop. If we pop on an empty stack or if the stack is not empty at the end, the expression is incorrect.

Answer:

```
expression ← "(3 + 2) × (5 - 1)"
stack ← []

FOR i FROM 1 TO length(expression) DO
char ← expression[i]
IF char = '(' THEN
```

```
push(stack, '(')
ELSE IF char = ')' THEN
IF stack is empty THEN
DISPLAY "Error: closing parenthesis without opening"
STOP
ELSE
pop(stack)
END IF
END IF
END FOR

IF stack is empty THEN
DISPLAY "Expression correct"
ELSE
DISPLAY "Error: missing ", size(stack), " closing parenthesis(es)"
END IF
```

Self-assessment Quiz

1. A stack follows the principle:

a) FIFO b) LIFO ✓ c) LILO

2. The suitable structure for a print queue is:

a) A stack b) A queue ✓ c) An array

3. In an array, accessing an element by its index has a complexity of:

a) O(1) ✓ b) O(n) c) O(n^2)

4. The "Undo" function (Ctrl+Z) of an editor uses:

a) A queue b) A stack ✓ c) An array

5. Which structure is used to implement a recursive algorithm?

a) Queue b) Stack ✓ c) Linked list

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Stack LIFO — operations	20%	Incorrect operations	Partially correct operations	Correct operations
Queue FIFO — operations	20%	Incorrect operations	Partially correct operations	Correct operations
Stack simulation	20%	Inaccurate simulation	Partial simulation	Correct simulation
Queue simulation	20%	Inaccurate simulation	Partial simulation	Correct simulation
Choosing the right structure	20%	No justified choices	1-2 justified choices	3 justified choices

Pedagogical Note ENS

Teaching transposition

Stacks and queues are very visual and concrete notions. In middle/high school, they can be introduced with physical objects: a stack of plates (LIFO) and a queue at the cafeteria (FIFO). Students can play the role of operations (push/pop, enqueue/dequeue) by lining up.

The parentheses algorithm is a classic in NSI Terminale.

Trainer advice

For exercises 2 and 3, have students act out the simulations at the board with volunteers: one student is the stack, another is the operator. Each push/pop is materialized with a sticky note on the board. Physical visualization helps understand the difference between LIFO and FIFO. Exercise 4 (choosing a structure) is excellent for collective discussion: each proposal can be debated.

Extension

Extend exercise 5 to handle three types of brackets: `()`, `[]`, `{}`. The algorithm must check matching: a closing bracket must correspond to the last opening bracket of the same type. Implement this algorithm in Python (`parentheses.py`) with a `Stack` class.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_07.pdf`

Computer Science Tutorial — ENS Year 1 — Session 07 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student is able to:

• Distinguish the different types of networks (LAN, WAN, Internet) and their scope

• Understand the role of IP addresses (IPv4/IPv6), subnet mask, and DNS

• Describe the steps of a client-server communication and the role of each protocol

• Associate network protocols with their default ports

• Compare file sharing methods and recommend the most suitable one

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — Network vocabulary	15 min	20 min
Exercise 2 — IP addresses	20 min	40 min
Exercise 3 — Client-server	15 min	55 min
Exercise 4 — Protocols and ports	20 min	75 min
Exercise 5 — Cloud and storage	15 min	90 min
Total	90 min	

Types of networks and client-server communication

```
graph TB
subgraph "Types of networks"
LAN["LAN — Local Area Network<br/>(room, building)"]
WAN["WAN — Wide Area Network<br/>(city, country)"]
Internet["Internet<br/>Worldwide network of networks"]
end

LAN <--> WAN
WAN <--> Internet

subgraph "Client-server communication"
Client[Browser<br/>Client] --> DNS[DNS Server<br/>Name → IP]
Client --> Server[Web Server]
Server --> Client
end
```

```
Internet --> Client

style LAN fill:#2ecc71,color:#fff
style WAN fill:#e67e22,color:#000
style Internet fill:#4a90d9,color:#fff
style Client fill:#9b59b6,color:#fff
style DNS fill:#e74c3c,color:#fff
style Server fill:#e74c3c,color:#fff
```

Protocols and ports

```
graph LR
subgraph "Application Layer"
HTTP["HTTP :80<br/>Unencrypted browsing"]
HTTPS["HTTPS :443<br/>Encrypted browsing"]
FTP["FTP :21<br/>File transfer"]
SSH["SSH :22<br/>Secure remote access"]
SMTP["SMTP :25<br/>Sending emails"]
IMAP["IMAP :143<br/>Receiving emails"]
end

subgraph "Transport Layer"
TCP["TCP<br/>Reliable connection"]
UDP["UDP<br/>Fast, unreliable"]
end

HTTP --> TCP
HTTPS --> TCP
FTP --> TCP
SSH --> TCP
SMTP --> TCP
IMAP --> TCP

style HTTP fill:#4a90d9,color:#fff
style HTTPS fill:#2ecc71,color:#fff
style FTP fill:#e67e22,color:#000
style SSH fill:#9b59b6,color:#fff
style SMTP fill:#e74c3c,color:#fff
style IMAP fill:#e74c3c,color:#fff
style TCP fill:#4a90d9,color:#fff
style UDP fill:#4a90d9,color:#fff
```

Exercise 1 — Network vocabulary (15 min)

Problem statement: Match each term with its definition.

Term	Definition
1. LAN	A. Worldwide network interconnecting millions of networks
2. WAN	B. Unique address of a device on the network
3. Internet	C. Local network (building, room)
4. IP Address	D. Translates domain names into IP addresses
5. DNS	E. Wide area network (city, country)
6. URL	F. Address of a resource on the Web

Answer: 1→C, 2→E, 3→A, 4→B, 5→D, 6→F

Exercise 2 — IP addresses (20 min)

Problem statement:

1. What is the difference between IPv4 and IPv6?
2. Which class does the address `192.168.1.1` belong to?
3. How many different IPv4 addresses can be encoded at most?
4. How many IPv6 addresses?
5. What does the command `ping 8.8.8.8` do?

Answer:

1. IPv4: 32 bits (4 bytes), ~4.3 billion addresses. IPv6: 128 bits, astronomical number of addresses.
2. `192.168.x.x` is a private class C address (local networks).
3. 2^{32} = 4,294,967,296 (approximately 4.3 billion).
4. 2^{128} = 340,282,366,920,938,463,463,374,607,431,768,211,456 (340 undecillion).
5. `ping 8.8.8.8` tests connectivity to Google's DNS server — if it responds, the Internet connection works.

Exercise 3 — Client-server (15 min)

Problem statement: Describe the communication steps when a user visits `https://www.ens-mostaganem.dz`.

For each step, indicate whether it is the client (browser) or the server that acts.

1. Entering the URL in the address bar
2. DNS request to find the IP address
3. Establishing the TCP connection
4. TLS negotiation (encryption)
5. Sending the HTTP GET request
6. Processing the request by the server
7. Sending the response (HTML page)
8. Displaying the page in the browser

Answer:

1. Client (user)
2. Client (the browser queries the DNS)
3. Client and server (TCP handshake)
4. Client and server (TLS handshake)
5. Client

6. Server

7. Server

8. Client

Exercise 4 — Protocols and ports (20 min)

Problem statement: Match each protocol with its default port and its use.

Protocol	Port	Use
HTTP	22	Secure file transfer
HTTPS	80	Secure remote access
FTP	21	Unencrypted web browsing
SSH	443	Encrypted web browsing
SMTP	25	Receiving emails
IMAP	143	Sending emails

Answer:

Protocol	Port	Use
HTTP	80	Unencrypted web browsing
HTTPS	443	Encrypted web browsing
FTP	21	File transfer
SSH	22	Secure remote access
SMTP	25	Sending emails
IMAP	143	Receiving emails (multi-device sync)

Exercise 5 — Cloud and storage (15 min)

Problem statement: You need to share a 50 MB folder (courses, tutorials, images) with your tutorial group.

1. What are the advantages and disadvantages of each method:

a) USB key

b) Email (attachment)

c) Google Drive / OneDrive

d) Moodle

2. Which method do you recommend and why?

Answer:

Method	Advantages	Disadvantages
USB key	No Internet needed, fast	Risk of loss/virus, in-person only
Email	Simple, traceable	Attachment limit (often 25 MB)

Drive/OneDrive	Share via link, accessible everywhere, versioning	Requires Internet and an account
Moodle	Integrated with the university, secure	Less flexible, interface sometimes heavy

Recommendation: Drive or Moodle for 50 MB (email is limited to 25 MB). USB key as a backup for in-person.

Self-assessment Quiz

1. A LAN network covers:

a) A city b) A building or room ✓ c) The whole world

2. IPv4 uses addresses of:

a) 32 bits ✓ b) 64 bits c) 128 bits

3. The HTTPS protocol uses port:

a) 80 b) 443 ✓ c) 22

4. The role of DNS is to:

a) Encrypt communications b) Translate a domain name into an IP address ✓ c) Manage emails

5. Which protocol is used to send an email?

a) HTTP b) FTP c) SMTP ✓

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Network types (LAN/WAN)	15%	Does not distinguish types	1-2 correct types	3 correct type
IP addressing and mask	20%	Incorrect calculations	Partially correct calculations	Correct calcu
Client-server model	20%	Does not understand the model	Partial understanding	Complete und
Protocols and ports	20%	No protocol identified	2-3 protocols	5 correct prot
Cloud and storage	15%	Does not know the services	1-2 services cited	3+ services w
Network diagram	10%	No diagram	Partial diagram	Complete and

Pedagogical Note ENS

Teaching transposition

Network concepts can be introduced in SNT Seconde with simple analogies: the local network is like the rooms of a school connected by hallways, the Internet is like the global road network. The IP address is like a postal address, DNS is like a phone book. For protocols, HTTP can be compared to a postcard (plain text) and HTTPS to a sealed letter.

Trainer advice

Exercise 3 (client-server) is central: ask students to re-enact the scene in pairs, one playing the client and the other the server. Each step is verbalized. For exercise 4 (ports), have

them create a laminated memory card with protocols/ports to review regularly. Exercise 5 (Cloud) is very concrete: have students share their own file sharing practices.

Extension

Set up a small local web server with Python (`python -m http.server 8000`) and access it from a browser. Observe the requests with developer tools (F12 → Network tab). Identify HTTP codes (200, 404, 500) and understand what they mean.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_08.pdf`

Computer Science Tutorial — ENS Year 1 — Session 08 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student is able to:

• Identify the three pillars of security (Confidentiality, Integrity, Availability)

• Evaluate password strength and apply best practices

• Recognize the signs of a phishing attempt

• Apply the 3-2-1 backup rule

• Identify situations that comply or not with the GDPR

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — CIA Triangle	15 min	20 min
Exercise 2 — Passwords	20 min	40 min
Exercise 3 — Phishing	20 min	60 min
Exercise 4 — 3-2-1 Backup	15 min	75 min
Exercise 5 — GDPR	15 min	90 min
Total	90 min	

CIA Triangle of security

```
graph TB
subgraph "CIA Triangle"
C[Confidentiality<br/>Only authorized people<br/>can access the data]
I[Integrity<br/>Data has not been<br/>modified without authorization]
D[Availability<br/>Data is accessible<br/>when needed]
end

C <--> I
I <--> D
D <--> C

C -.-> Ex1["🔒 Ransomware: encrypted files"]
I -.-> Ex2["✏️ Grade modified without authorization"]
D -.-> Ex3["🚫 Moodle server inaccessible"]
```

```
style C fill:#4a90d9,color:#fff
style I fill:#2ecc71,color:#fff
style D fill:#e67e22,color:#000
style Ex1 fill:#e74c3c,color:#fff
style Ex2 fill:#e74c3c,color:#fff
style Ex3 fill:#e74c3c,color:#fff
```

Phishing detection

```
flowchart TD
A[Email received] --> B{Legitimate<br/>sender?}
B -->|Yes| C{Urgency<br/>requested?}
B -->|No| Phish[ PHISHING]
C -->|Yes| D{Request for<br/>credentials?}
C -->|No| E{Suspicious<br/>URL?}
D -->|Yes| Phish
D -->|No| F{Shortened URL<br/>bit.ly, tinyurl?}
E -->|Yes| Phish
E -->|No| Safe[ Legitimate email]
F -->|Yes| Phish
F -->|No| Safe

style A fill:#4a90d9,color:#fff
style Safe fill:#2ecc71,color:#fff
style Phish fill:#e74c3c,color:#fff
style B fill:#e67e22,color:#000
style C fill:#e67e22,color:#000
style D fill:#e67e22,color:#000
style E fill:#e67e22,color:#000
style F fill:#e67e22,color:#000
```

Exercise 1 — CIA Triangle (15 min)

Problem statement: Identify the security pillar (Confidentiality, Integrity, Availability) involved in each situation:

1. A ransomware encrypts a teacher's files.
2. A student modifies their grade in the class Excel file.
3. The Moodle server is inaccessible on the day tutorial submissions are due.
4. An email containing passwords is intercepted.
5. A hard drive fails without a backup.

Answer:

1. Confidentiality + Availability (files inaccessible and encrypted)
2. Integrity (data modified without authorization)
3. Availability (service unavailable)
4. Confidentiality (intercepted data)
5. Availability (lost data)

Exercise 2 — Passwords (20 min)

Problem statement: Rank the following passwords from weakest to strongest. Justify.

1. `123456`
2. `Mot2Passe!`
3. `ENS.Mostaganem2026!`
4. `azerty`
5. `J'aimeLesTartesAuxPommes!`
6. `password`

Answer (from weakest to strongest):

Rank	Password	Length	Issue
6 (weak)	123456	6	Sequential, top 1 passw
5	password	8	Common word, dictiona
4	azerty	6	French dictionary word
3	Mot2Passe!	10	Compound word with p
2	J'aimeLesTartesAuxPommes!	27	Very long but full sente
1 (strong)	ENS.Mostaganem2026!	22	Long, mixed case/numb

Rule: min 12 characters, mix of types, no dictionary word.

Exercise 3 — Phishing (20 min)

Problem statement: Analyze the following email and identify 5 warning signs:

```
From: support@enseignemrnt-mosta.dz
Subject: URGENT — Your account will be closed

Hello,

Your university email account will be deactivated within 48 hours
due to lack of update. Click here to confirm your credentials:
http://bit.ly/2XkL9pQ

Sincerely,
The IT department
ENS Mostaganem
```

Answer — Warning signs:

1. Suspicious sender: `enseignemrnt-mosta.dz` (spelling mistake, not the official domain)
2. Urgency: "URGENT", "48 hours" — psychological pressure to avoid thinking
3. Request for credentials: no legitimate service asks for a password by email
4. Shortened URL: `bit.ly` hides the real destination
5. Vague signature: "The IT department" with no specific name

What to do: Do not click. Report to administration. Check directly on the official website.

Exercise 4 — 3-2-1 Backup (15 min)

Problem statement: Apply the 3-2-1 rule for a teacher who has:

• Their courses on their laptop

• Their grades on a USB key

• Their thesis on the office hard drive

Propose a concrete backup strategy.

Answer:

Copy	Medium	Location
Original	Laptop	Office
Copy 1	External hard drive	Home (in a drawer)
Copy 2	Cloud (OneDrive/Google Drive)	Online

3 copies (original + 2 backups), 2 different media (hard drive + cloud), 1 copy off-site (cloud).

For the thesis: additionally, paper version + USB key in a different location (at a colleague's).

Exercise 5 — GDPR and ethics (15 min)

Problem statement: For each situation, say whether it complies with GDPR or not, and why.

1. A teacher posts a list of grades with first and last names in the hallway.

2. A teacher uses their personal Gmail email to contact their students.

3. A teacher deletes student files at the end of the school year.

4. A teacher films their students in class without informing them.

5. A teacher uses student data for research without anonymizing it.

Answer:

1. Not compliant — grades are personal data, public display is prohibited without consent.

2. Not compliant — personal emails (Gmail, Yahoo) do not guarantee data protection. Use the institutional email.

3. Compliant — GDPR requires a retention period limited to what is necessary.

4. Not compliant — the right to image requires prior written authorization (for minors, parental consent).

5. Not compliant — data must be anonymized before use in research.

Self-assessment Quiz

1. The security pillar that guarantees data has not been modified is:

a) Confidentiality b) Integrity ✓ c) Availability

2. A strong password must:

a) Be short and easy to remember b) Contain at least 12 mixed characters ✓ c) Contain your first name

3. An email asking you to click urgently to confirm your password is:

a) A legitimate message from IT b) A phishing attempt ✓ c) An automated email

4. The 3-2-1 backup rule means:

a) 3 copies, 2 media, 1 off-site ✓ b) 3 backups per day c) 2 USB keys and 1 hard drive

5. According to GDPR, personal data must be:

a) Kept indefinitely b) Anonymized for research ✓ c) Shared on social networks

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
CIA Triad	20%	Does not know the 3 pillars	2 out of 3 pillars	3 correct pillars w
Strong passwords	20%	Cannot distinguish strong/weak	Criteria partially identified	Correct criteria +
Phishing	20%	No sign identified	1-2 signs identified	3+ correct signs
Workstation security	20%	No measure proposed	1-2 measures	3+ relevant mea
GDPR and ethics	20%	Does not know GDPR	Principles partially understood	Principles well un

Pedagogical Note ENS

Teaching transposition

Digital security is a theme in the SNT (Seconde) curriculum and a major citizenship issue. The CIA triangle can be taught with everyday life examples: confidentiality is a locked diary, integrity is a document whose integrity is verified, availability is being able to access your notebook when needed. Phishing can be the subject of a practical workshop with fake emails prepared by the teacher.

Trainer advice

Exercise 2 (passwords) is very revealing: ask students to test their own passwords on sites like "howsecureismypassword.net" (being careful not to enter real passwords). Exercise 3 (phishing) can be turned into a game: prepare 5 emails (3 legitimate and 2 phishing) and ask students to sort them. Exercise 5 (GDPR) is crucial for the civic education of future teachers.

Extension

Organize a "Capture The Flag" (CTF) challenge adapted for beginners: students must identify phishing emails, find weak passwords in scenarios, and propose corrections. Use the "Root-Me" platform or a home-made CTF prepared by the teacher with basic security challenges.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_09.pdf`

Computer Science Tutorial — ENS Year 1 — Session 09 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student is able to:

• Implement search algorithms (sequential, binary) in Python

• Implement simple sorting algorithms (selection sort, bubble sort)

• Advanced Python list manipulation (list comprehension, slicing, filtering)

• Analyze the algorithmic complexity of Python code

• Solve algorithmic problems using the data structures seen in class

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome and algorithmic reminders	5 min	5 min
Exercise 1 — Search	20 min	25 min
Exercise 2 — Selection sort	20 min	45 min
Exercise 3 — Stack and parentheses	20 min	65 min
Exercise 4 — Lists and filtering	15 min	80 min
Exercise 5 — Merge sort	10 min	90 min
Total	90 min	

Comparison of search algorithms

```
flowchart TB
subgraph "Sequential search"
direction LR
S1[Start] --> S2[Browse<br/>element by element]
S2 --> S3{Found?}
S3 -->|Yes| S4[Return index]
S3 -->|No| S5[End of array?]
S5 -->|No| S2
S5 -->|Yes| S6[Return -1]
end

subgraph "Binary search"
direction LR
D1[Start] --> D2[Sorted array?]
D2 --> D3[Look at the middle]
```

```
D3 --> D4{Found?}
D4 -->|Yes| D5[Return index]
D4 -->|No| D6{Greater<br/>or smaller?}
D6 -->|Greater| D7[Search on the right]
D6 -->|Smaller| D8[Search on the left]
D7 --> D3
D8 --> D3
end

S6 --> Comp[Comparison<br/>Sequential: O(n)<br/>Binary: O(log n)]
D5 --> Comp

style S1 fill:#4a90d9,color:#fff
style S4 fill:#2ecc71,color:#fff
style S6 fill:#e74c3c,color:#fff
style D1 fill:#4a90d9,color:#fff
style D5 fill:#2ecc71,color:#fff
style Comp fill:#9b59b6,color:#fff
```

Complexity of sorting algorithms

```
graph LR
subgraph "Sorting algorithms"
TS[Selection sort<br/>O(n²)]
TB[Bubble sort<br/>O(n²)]
TF[Merge sort<br/>O(n log n)]
TQ[Quicksort<br/>O(n log n)]
end

subgraph "Performance (n = 1000)"
TS --> T1["~500,000 comparisons"]
TB --> T2["~500,000 comparisons"]
TF --> T3["~10,000 comparisons"]
TQ --> T4["~10,000 comparisons"]
end

TS -.->|"Simple to implement<br/>But slow"| Note1
TF -.->|"More complex<br/>But fast"| Note2

style TS fill:#e74c3c,color:#fff
style TB fill:#e74c3c,color:#fff
style TF fill:#2ecc71,color:#fff
style TQ fill:#2ecc71,color:#fff
style T1 fill:#e67e22,color:#000
style T2 fill:#e67e22,color:#000
style T3 fill:#e67e22,color:#000
style T4 fill:#e67e22,color:#000
```

Exercise 1 — Sequential and binary search (20 min)

Problem statement: Create a script `recherche.py` that implements two search algorithms:

1. `sequential_search(list, value)` — traverses the list and returns the index or -1

2. `binary_search(list, value)` — searches in a sorted list (divide and conquer)

For each algorithm, count the number of comparisons made and compare them.

Answer:

```
def sequential_search(list, value):
"""Sequential search. Returns the index or -1."""
```

```
comparisons = 0
for i, v in enumerate(list):
comparisons += 1
if v == value:
print(f" Sequential: {comparisons} comparison(s)")
return i
print(f" Sequential: {comparisons} comparison(s)")
return -1

def binary_search(list, value):
"""Binary search (sorted list). Returns the index or -1."""
comparisons = 0
start, end = 0, len(list) - 1
while start <= end:
comparisons += 1
mid = (start + end) // 2
if list[mid] == value:
print(f" Binary: {comparisons} comparison(s)")
return mid
elif list[mid] < value:
start = mid + 1
else:
end = mid - 1
print(f" Binary: {comparisons} comparison(s)")
return -1

# Test
data = [2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78, 89, 92, 99, 101]
print(f"List: {data}")
print(f"Search for 67:")
print(f" Index found: {sequential_search(data, 67)}")
print(f" Index found: {binary_search(data, 67)}")

print(f"\nSearch for 3 (not found):")
print(f" Index found: {sequential_search(data, 3)}")
print(f" Index found: {binary_search(data, 3)}")
```

Pedagogical question: On a list of 1,000,000 elements, how many comparisons at most for each method? (Answer: sequential = 1,000,000, binary = $\log_2(1{,}000{,}000) \approx 20$)

Exercise 2 — Selection sort (20 min)

Problem statement: Create a script `selection_sort.py` that implements selection sort. The program must:

1. Implement `selection_sort(list)` that sorts the list in place
2. Display the list before and after sorting
3. Count the number of swaps and comparisons
4. Test on a randomly generated list of 10 integers

Principle reminder: At each iteration, look for the minimum in the unsorted part and place it at the beginning of that part.

Answer:

```
import random

def selection_sort(list):
"""Selection sort. Returns the number of swaps and comparisons."""
n = len(list)
```

```
swaps = 0
comparisons = 0
for i in range(n - 1):
min_idx = i
for j in range(i + 1, n):
comparisons += 1
if list[j] < list[min_idx]:
min_idx = j
if min_idx != i:
list[i], list[min_idx] = list[min_idx], list[i]
swaps += 1
return swaps, comparisons

# Test
list = [random.randint(0, 100) for _ in range(10)]
print(f"Original list: {list}")
sw, comp = selection_sort(list)
print(f"Sorted list : {list}")
print(f"Swaps: {sw}, Comparisons: {comp}")
print(f"Complexity: O(n²) with n = {len(list)} → ~{len(list)**2 // 2} comparisons")
```

Exercise 3 — Stack with parentheses validation (20 min)

Problem statement: Create a script `parentheses.py` that extends the parentheses validation algorithm seen in class. The program must:

1. Implement a `Stack` class with methods `push()`, `pop()`, `is_empty()`, `size()`

2. Use this class to validate expressions with `()`, `[]`, `{}`

3. In case of error, indicate precisely where the problem is (character position)

Answer:

```
class Stack:
"""Implementation of a LIFO stack."""
def __init__(self):
self._elements = []

def push(self, element):
self._elements.append(element)

def pop(self):
if not self.is_empty():
return self._elements.pop()
return None

def peek(self):
if not self.is_empty():
return self._elements[-1]
return None

def is_empty(self):
return len(self._elements) == 0

def size(self):
return len(self._elements)

def validate_parentheses(expression):
"""Validates parentheses and returns (valid, error_position, message)."""
matching = {")": "(", "]": "[", "}": "{"}
opening = "([{"
closing = ")]}"
```

```
stack = Stack()

for i, char in enumerate(expression):
if char in opening:
stack.push(char)
elif char in closing:
if stack.is_empty():
return False, i, f"Closing bracket '{char}' without opening at position {i}"
top = stack.pop()
if top != matching[char]:
return False, i, f"Mismatch: '{top}' opened but '{char}' closed at position {i}"

if not stack.is_empty():
return False, len(expression) - 1, f"Missing {stack.size()} closing bracket(s)"

return True, -1, "Expression is correct"

# Tests
expressions = [
"(3 + 2) × [5 - {1 + 2}]",
"(3 + 2] × [5 - {1 + 2})",
"((1 + 2) × 3)",
"({[)]}",
"((({}))"
]

for expr in expressions:
valid, pos, msg = validate_parentheses(expr)
status = "✓" if valid else "✗"
print(f"{status} {expr[:50]:50s} → {msg}")
```

Exercise 4 — List algorithms (filtering and transformation) (15 min)

Problem statement: Create a script `filtering.py` that, from a list of random grades (20 grades between 0 and 20):

1. Computes the mean, median, and standard deviation manually (without the statistics module)

2. Filters to get grades ≥ 10 (passing)

3. Creates a new list with grades increased by 1 point (bonus, capped at 20)

4. Uses list comprehensions

Answer:

```
import random
import math

grades = [random.randint(0, 20) for _ in range(20)]
print(f"Original grades: {grades}")

# 1. Manual statistics
mean = sum(grades) / len(grades)
sorted_grades = sorted(grades)
n = len(sorted_grades)
median = (sorted_grades[n // 2 - 1] + sorted_grades[n // 2]) / 2 if n % 2 == 0 else
sorted_grades[n // 2]
variance = sum((x - mean) ** 2 for x in grades) / n
std_dev = math.sqrt(variance)
```

```
print(f"Mean : {mean:.2f}")
print(f"Median : {median:.2f}")
print(f"Std. Dev. : {std_dev:.2f}")

# 2. Grades ≥ 10
passing = [g for g in grades if g >= 10]
print(f"Passing (≥10): {passing} ({len(passing)}/{len(grades)})")

# 3. Capped bonus
bonus = [min(g + 1, 20) for g in grades]
print(f"With bonus : {bonus}")
```

Exercise 5 — Merge sort algorithm (recursive) (10 min)

Problem statement: Create a script `merge_sort.py` that implements merge sort, a recursive algorithm of complexity O(n log n).

Principle: divide the list in two, recursively sort each half, then merge the two sorted halves.

Answer:

```
import random

def merge_sort(list):
"""Recursive merge sort."""
if len(list) <= 1:
return list

mid = len(list) // 2
left = merge_sort(list[:mid])
right = merge_sort(list[mid:])

return merge(left, right)

def merge(left, right):
"""Merges two sorted lists."""
result = []
i = j = 0
while i < len(left) and j < len(right):
if left[i] <= right[j]:
result.append(left[i])
i += 1
else:
result.append(right[j])
j += 1
result.extend(left[i:])
result.extend(right[j:])
return result

# Test
list = [random.randint(0, 100) for _ in range(15)]
print(f"Original: {list}")
sorted_list = merge_sort(list)
print(f"Sorted : {sorted_list}")
print(f"Complexity: O(n log n) with n = {len(list)}")
```

Self-assessment Quiz

1. Binary search requires the list to be:

a) Unsorted b) Sorted ✓ c) Of fixed size

2. The complexity of selection sort is:

a) O(n) b) O(n log n) c) O(n²) ✓

3. In Python, the syntax `[x*2 for x in range(5)]` is:

a) A for loop b) A list comprehension ✓ c) A lambda function

4. Merge sort has a complexity of:

a) O(n²) b) O(n log n) ✓ c) O(log n)

5. Which stack method returns the top element without removing it?

a) `push()` b) `pop()` c) `peek()` ✓

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Sequential search	15%	Incorrect algorithm	Partial algorithm	Correct algorithm
Binary search	15%	Incorrect algorithm	Partial algorithm	Correct algorithm
Selection sort	20%	Incorrect sort	Partially correct sort	Correct sort
Stack — parentheses	20%	Incorrect algorithm	Partial algorithm	Correct algorithm
Merge sort	20%	Incorrect sort	Partially correct sort	Correct sort
Notion of complexity	10%	Complexity not identified	1-2 correct complexities	3+ correct complexities

Pedagogical Note ENS

Teaching transposition

This tutorial shows how algorithmic concepts (search, sort) and data structures (stack) seen in class are concretely implemented in Python. These algorithms are part of the NSI Terminale curriculum. Comparing complexities (O(n) vs O(log n) vs O(n²) vs O(n log n)) is a key objective of the curriculum. In NSI Première, one can start with sequential search and selection sort before tackling binary search and merge sort.

Trainer advice

Exercise 1 is fundamental: have students mentally execute both algorithms on a small dataset before coding. For exercise 2, ask students to trace the array evolution at each iteration on paper. Exercise 3 (parentheses) is an excellent example of the usefulness of stacks. Exercise 5 (merge sort) introduces recursion: mastering it is difficult but crucial for the rest of the curriculum.

Extension

Propose a challenge: implement quicksort in Python and compare its performance with merge sort on a list of 10,000 elements using the `time` module. Create a comparison table of execution times for different sorting algorithms and different data sizes.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_AN_10.pdf`

Computer Science Tutorial — ENS Year 1 — Session 10 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

By the end of this tutorial session, the student is able to:

• Implement a queue (FIFO) in Python with a class

• Solve algorithmic problems using queues and stacks

• Manipulate dictionaries and nested data structures

• Implement a simple breadth-first search (BFS) algorithm

• Apply data structures to concrete problems

Tutorial Plan

Phase	Duration	Cumulative Total
Welcome	5 min	5 min
Exercise 1 — Queue	20 min	25 min
Exercise 2 — RPN expression (stack)	20 min	45 min
Exercise 3 — BFS (queue + graph)	20 min	65 min
Exercise 4 — Student dictionaries	15 min	80 min
Exercise 5 — RLE compression	10 min	90 min
Total	90 min	

Evaluating an expression in Reverse Polish Notation (RPN)

```
flowchart TD
A[RPN expression: 5 3 8 + -] --> B[Split elements<br/>['5', '3', '8', '+', '-']]
B --> C[Initial stack: []]

C --> D1{Read '5'}
D1 -->|Number| E1[Push 5]
E1 --> D2{Read '3'}
D2 -->|Number| E2[Push 3]
E2 --> D3{Read '8'}
D3 -->|Number| E3[Push 8]
E3 --> D4{Read '+'}
D4 -->|Operator| F4[Pop 8 and 3]
F4 --> G4[Compute 3 + 8 = 11]
G4 --> H4[Push 11]
H4 --> D5{Read '-'}
```

```
D5 -->|Operator| F5[Pop 11 and 5]
F5 --> G5[Compute 5 - 11 = -6]
G5 --> H5[Push -6]
H5 --> I[Result: -6]

style A fill:#4a90d9,color:#fff
style I fill:#2ecc71,color:#fff
style E1 fill:#e67e22,color:#000
style E2 fill:#e67e22,color:#000
style E3 fill:#e67e22,color:#000
style G4 fill:#9b59b6,color:#fff
style G5 fill:#9b59b6,color:#fff
style H4 fill:#e67e22,color:#000
style H5 fill:#e67e22,color:#000
```

Breadth-First Search (BFS) of a graph

```
graph TD
ENTRANCE["Entrance"] --> HALL["Hall"]
HALL --> ROOM1["Room 1"]
HALL --> LIBRARY["Library"]
HALL --> OFFICE["Office"]
LIBRARY --> ROOM2["Room 2"]
OFFICE --> ROOM3["Room 3"]
ROOM1 --> ROOM2
ROOM2 --> ROOM3

style ENTRANCE fill:#e74c3c,color:#fff
style HALL fill:#e67e22,color:#000
style ROOM1 fill:#4a90d9,color:#fff
style LIBRARY fill:#4a90d9,color:#fff
style OFFICE fill:#4a90d9,color:#fff
style ROOM2 fill:#2ecc71,color:#fff
style ROOM3 fill:#9b59b6,color:#fff
```

Exercise 1 — Queue class (20 min)

Problem statement: Create a script `queue.py` that implements a `Queue` class with:

1. Methods `enqueue()`, `dequeue()`, `peek()`, `is_empty()`, `size()`

2. A `display()` method that shows the queue state

3. A test with a counter simulation: clients arrive and are served

Answer:

```
from collections import deque

class Queue:
"""Implementation of a FIFO queue."""
def __init__(self):
self._elements = deque()

def enqueue(self, element):
self._elements.append(element)

def dequeue(self):
if not self.is_empty():
return self._elements.popleft()
return None
```

```
def peek(self):
if not self.is_empty():
return self._elements[0]
return None

def is_empty(self):
return len(self._elements) == 0

def size(self):
return len(self._elements)

def display(self):
print(f"Queue: {list(self._elements)} (head → {self.peek()}, {self.size()} client(s))")

# Counter simulation
print("=== Counter simulation ===\n")
queue = Queue()

# Client arrival
for name in ["Ali", "Sara", "Mohamed", "Leila", "Ahmed"]:
queue.enqueue(name)
print(f"✓ {name} joins the queue.")

print(f"\nInitial state:")
queue.display()

# Service
print("\nService in progress:")
while not queue.is_empty():
client = queue.dequeue()
print(f"→ Service for {client} completed.")
if not queue.is_empty():
print(f" Next client: {queue.peek()}")

print("\n✓ All clients have been served.")
```

Exercise 2 — Postfix expression evaluation (RPN) (20 min)

Problem statement: Create a script `rpn.py` that evaluates an expression in Reverse Polish Notation (RPN) using a stack.

Principle: The operator follows its operands. Example: 3 4 + = 3 + 4 = 7

Examples:

- 3 4 + → 7
- 5 3 8 + - → 5 - (3 + 8) = -6
- 10 4 2 + * → 10 × (4 + 2) = 60

Answer:

```
def evaluate_rpn(expression):
"""Evaluates an expression in Reverse Polish Notation."""
stack = []
operators = {
"+": lambda a, b: a + b,
"-": lambda a, b: a - b,
"*": lambda a, b: a * b,
"/": lambda a, b: a / b if b != 0 else None
}
```

```
elements = expression.split()
for elem in elements:
if elem in operators:
if len(stack) < 2:
return "Error: not enough operands"
b = stack.pop()
a = stack.pop()
result = operators[elem](a, b)
if result is None:
return "Error: division by zero"
stack.append(result)
else:
try:
stack.append(float(elem))
except ValueError:
return f"Error: invalid element '{elem}'"

if len(stack) != 1:
return f"Error: {len(stack)} results instead of 1"
return stack[0]

# Tests
expressions = [
"3 4 +",
"5 3 8 + -",
"10 4 2 + *",
"15 7 1 1 + - / 3 * 2 1 1 + + -",
"4 0 /"
]

for expr in expressions:
result = evaluate_rpn(expr)
print(f"{expr:30s} → {result}")
```

Exercise 3 — Breadth-First Search of a graph (BFS) (20 min)

Problem statement: Create a script `bfs.py` that implements a Breadth-First Search using a queue. The graph represents a school floor plan: each room is connected to others by hallways.

```
graph TD
ENTRANCE --> HALL
HALL --> ROOM1
HALL --> LIBRARY
HALL --> OFFICE
LIBRARY --> ROOM2
OFFICE --> ROOM3
ROOM1 --> ROOM2
ROOM2 --> ROOM3
```

Answer:

```
from collections import deque

def bfs(graph, start):
"""Breadth-First Search from a node."""
visited = set()
queue = deque()
order = []

queue.append(start)
visited.add(start)
```

```
print(f"Breadth-First Search from '{start}':")

while queue:
current = queue.popleft()
order.append(current)
print(f" Visiting: {current}")

for neighbor in sorted(graph[current]):
if neighbor not in visited:
visited.add(neighbor)
queue.append(neighbor)

return order

# Graph: school floor plan
school = {
"Entrance": ["Hall"],
"Hall": ["Entrance", "Room1", "Library", "Office"],
"Room1": ["Hall", "Room2"],
"Room2": ["Library", "Room1", "Room3"],
"Room3": ["Office", "Room2"],
"Library": ["Hall", "Room2"],
"Office": ["Hall", "Room3"]
}

visit_order = bfs(school, "Entrance")
print(f"\nVisit order: {' → '.join(visit_order)}")

# Question: what is the distance (number of doors) between the entrance and room 3?
# Answer: Entrance → Hall → Office → Room3 (3 doors)
```

Exercise 4 — Student dictionary with multi-criteria search (15 min)

Problem statement: Create a script `students.py` that manages a student database (list of dictionaries). The program:

1. Creates a list of at least 5 students with: name, grades (list), group, major
2. Computes the average for each student
3. Provides search functions by group, by major, by average
4. Sorts students by descending average

Answer:

```
def create_student(name, group, major, grades):
"""Creates a student dictionary."""
average = sum(grades) / len(grades) if grades else 0
return {
"name": name,
"group": group,
"major": major,
"grades": grades,
"average": round(average, 2)
}

def search_by_group(students, group):
"""Searches for students in a group."""
return [s for s in students if s["group"] == group]
```

```
def search_by_major(students, major):
"""Searches for students in a major."""
return [s for s in students if s["major"] == major]

def search_by_average(students, threshold):
"""Searches for students with average ≥ threshold."""
return [s for s in students if s["average"] >= threshold]

def sort_by_average(students):
"""Sorts by descending average."""
return sorted(students, key=lambda s: s["average"], reverse=True)

# Create database
students = [
create_student("Ali", "G1", "French", [15, 12, 18]),
create_student("Sara", "G1", "French", [18, 16, 17]),
create_student("Mohamed", "G2", "English", [10, 11, 9]),
create_student("Leila", "G2", "French", [16, 14, 19]),
create_student("Ahmed", "G1", "English", [8, 10, 12]),
create_student("Fatima", "G2", "English", [14, 15, 13])
]

print("=== All students ===")
for s in students:
print(f"{s['name']:<10} {s['group']:<4} {s['major']:<10} Grades: {s['grades']} Avg:
{s['average']}")

print(f"\n=== Group G1 ===")
for s in search_by_group(students, "G1"):
print(f" {s['name']} — {s['major']} — Avg: {s['average']}")

print(f"\n=== Average ≥ 14 ===")
for s in search_by_average(students, 14):
print(f" {s['name']} — {s['average']}")

print(f"\n=== Overall ranking ===")
for i, s in enumerate(sort_by_average(students), 1):
print(f"{i:2d}. {s['name']:<10} {s['average']:.2f}")
```

Exercise 5 — Dictionary compression (run-length encoding) (10 min)

Problem statement: Create a script `rle.py` that implements a simple compression algorithm:

- RLE (Run-Length Encoding) replaces repetitions with (value, count)
- Example: AAABBBCCDAA → `[('A',3), ('B',3), ('C',2), ('D',1), ('A',2)]`
- Implement `compress(string)` and `decompress(data)`

Answer:

```
def compress(string):
"""Compresses a string with RLE."""
if not string:
return []

result = []
current = string[0]
count = 1

for c in string[1:]:
```

```
    if c == current:
    count += 1
    else:
    result.append((current, count))
    current = c
    count = 1

    result.append((current, count))
    return result

    def decompress(data):
    """Decompresses RLE data."""
    return "".join(c * n for c, n in data)

    # Tests
    example = "AAABBBCCDAA"
    print(f"Original : {example}")
    compressed = compress(example)
    print(f"Compressed : {compressed}")
    print(f"Ratio: {len(str(compressed)) / len(example) * 100:.1f}% of original size")
    decompressed = decompress(compressed)
    print(f"Decompressed : {decompressed}")
    print(f"Identical? {example == decompressed}")

    # Another test
    text = "WWWWWWWWWWWWBWWWWWWWWWWWWBBBWWWWWWWWWWWWWWWWWWWWWWWWB"
    comp = compress(text)
    print(f"\nOriginal text: {len(text)} characters")
    print(f"Compressed text: {len(comp)} pairs")
    print(f"Compression ratio: {len(comp) / len(text) * 100:.1f}%")
```

Self-assessment Quiz

1. A queue (FIFO) uses the principle:

a) Last in, first out b) First in, first out ✓ c) Random access

2. In Reverse Polish Notation, the expression 5 3 - equals:

a) 2 ✓ b) -2 c) 8

3. Breadth-First Search (BFS) uses a structure of:

a) Stack b) Queue ✓ c) Array

4. In Python, a dictionary is a data structure that:

a) Stores elements indexed by integers b) Associates keys with values ✓ c) Stores elements in insertion order

5. RLE (Run-Length Encoding) is useful for compressing:

a) Data with many repetitions ✓ b) Random data c) Already compressed files

Grading Rubric

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Stack/Queue class	20%	Missing implementation	Partial implementation	Correct implementation
Reverse Polish Notation	20%	Incorrect algorithm	Partial algorithm	Correct algorithm

Breadth-First Search	20%	Incorrect traversal	Partial traversal	Correct traversal
RLE compression	20%	Incorrect compression	Partial compression	Correct compression
User interface	20%	No interface	Partial interface	Functional interface

Pedagogical Note ENS

Teaching transposition

This synthesis tutorial mobilizes all acquired skills: classes, stacks, queues, dictionaries, algorithms. Reverse Polish Notation (exercise 2) is an excellent example of the usefulness of stacks in computing. Breadth-First Search (exercise 3) introduces graphs, a fundamental concept for teaching NSI Terminale. RLE compression (exercise 5) is intuitive and visual: students quickly understand the principle.

Trainer advice

Exercise 2 (RPN) is often a source of confusion because students must read the expression differently. Have students compute several RPN expressions "by hand" on the board before coding. For exercise 3 (BFS), use a large sheet of paper or the board to physically simulate the traversal: place sticky notes representing rooms and connect them with strings, then move a marker. Exercise 5 (RLE) is fun: ask students to find examples of repetitions in their own name or in sentences.

Extension

Propose a synthesis mini-project: create a Hangman game in Python that uses a `SecretWord` class with the notions of dictionary, list, and search algorithms seen in this tutorial. The game must draw a random word, hide the letters, and allow the user to propose letters until they have won or lost.